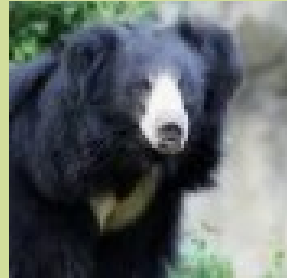


NEO4j Final Project

Group Sloth Bear: Project #3



Andrew H | Cindy T | Hannah R

Goal for the Spotify Music Graph

Goal 1: Build a music Knowledge graph

- Large Data Set

Goal 2: Analyze relationship between tracks, artists, and genre

- Good fit for group database
- Real world music analysis

Tools we used



Python

Data Cleaning



Neo4j

Group database

Spotify Music Graph

Using a Kaggle dataset, we cleaned the music data with Python and created csv files to be imported into NEO4j. From here, we ran a series of Cypher queries to test our data. These were each run 10 times and timed.



Data Cleaning

- Removed missing values
- Selected key columns
- Removed duplicates (track_id)
- Split multiple artists

```
df["artists"] = df["artists"].apply(normalize_artists)
df = df[df["artists"] != ""]

df_tracks = df[
    [
        "track_id",
        "track_name",
        "popularity",
        "danceability",
        "valence",
        "acousticness",
        "album_name",
        "track_genre",
        "artists"
    ]
].drop_duplicates(subset=["track_id"]).copy()

# Artist list for splitting
df_tracks["artist_list"] = df_tracks["artists"].astype(str).str.split()

# Final safety cleanup for artist_list
df_tracks["artist_list"] = df_tracks["artist_list"].apply(
    lambda lst: [a.strip() for a in lst if str(a).strip() != ""]
)

# Remove rows with no usable artists after splitting
df_tracks = df_tracks[df_tracks["artist_list"].map(len) > 0].copy()
```

CSV Setup

CSV Files Created:

- tracks.csv
- artists.csv
- albums.csv
- genres.csv

Relationships:

- performed_by
- belongs_to
- has_genre

```
# 8. Create albums.csv
# -----
albums = df_tracks[["album_name"]].drop_duplicates().copy()
albums["album_name"] = albums["album_name"].astype(str).str.strip()
albums = albums[albums["album_name"] != ""]
albums.rename(columns={"album_name": "name"}, inplace=True)
albums.to_csv("albums.csv", index=False)

# -----
# 9. Create genres.csv
# -----
genres = df_tracks[["track_genre"]].drop_duplicates().copy()
genres["track_genre"] = genres["track_genre"].astype(str).str.strip()
genres = genres[genres["track_genre"] != ""]
genres.rename(columns={"track_genre": "name"}, inplace=True)
genres.to_csv("genres.csv", index=False)

# -----
# 10. Create rel_performed_by.csv
# -----
performed_rows = []

for _, row in df_tracks.iterrows():
    for artist in row["artist_list"]:
        artist = artist.strip()
        if artist:
            performed_rows.append({
                "track_id": row["track_id"],
                "artist_name": artist
            })

rel_performed_by = pd.DataFrame(performed_rows).drop_duplicates()
rel_performed_by.to_csv("rel_performed_by.csv", index=False)
```

Workflow of SpotifyDB



Python



CSV



Neo4j

Queries

Spotifydb

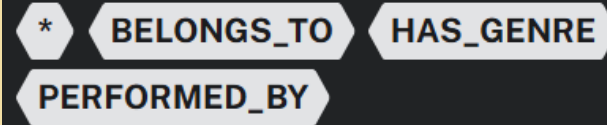
After cleaning the Kaggle dataset with python and importing the CSV files, we created nodes and relationships in our "spotifydb" database

Database information

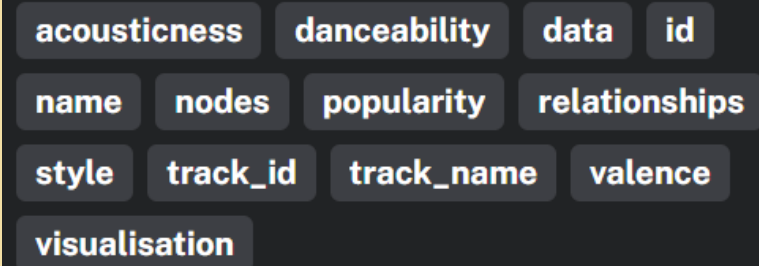
Nodes



Relationships



Property keys



Task 1

Highly Popular Tracks

Avg Run Time of 10: 14.5

Finding the top 20 tracks in the dataset

```
MATCH (t:Track)
WHERE t.popularity > 80
RETURN t.track_name AS track_name,
t.popularity AS popularity
ORDER BY t.popularity DESC
LIMIT 20;
```

	track_name	popularity
1	"Unholy (feat. Kim Petras)"	100
2	"Quevedo: Bzrp Music Sessions, Vol. 52"	99
3	"I'm Good (Blue)"	98
4	"La Bachata"	98
5	"Tití Me Preguntó"	97
6	"Me Porto Bonito"	97
7	"Under The Influence"	96
8	"Efecto"	96
9	"I Ain't Worried"	96

Task 2

Tracks with “Love” or “Dream”

Avg Run Time of 10: 39.1

Finding up to 50 tracks with titles containing the words “love” or “dream”

```
MATCH (t:Track)
WHERE toLower(t.track_name) CONTAINS 'love'
OR toLower(t.track_name) CONTAINS 'dream'
RETURN t.track_name AS track_name,
t.popularity AS popularity
ORDER BY t.popularity DESC
LIMIT 50;
```

	track_name	popularity
1	"Another Love"	93
2	"I Love You So"	89
3	"lovely (with Khalid)"	89
4	"Someone You Loved"	87
5	"Only Love Can Hurt Like This"	87
6	"LOVE DIVE"	85
7	"Leave Before You Love Me (with Jonas Brothers)"	85
8	"Lover"	85
9	"Crazy What Love Can Do"	85

Genre-wise track counts

Avg Run Time of 10: 24.2

Finding the number of tracks in each of the top 10 genres

```
MATCH (t:Track)-[:HAS_GENRE]->(g:Genre)
RETURN g.name AS genre_name,
COUNT(t) AS track_count
ORDER BY track_count DESC
LIMIT 10;
```

	genre_name	track_count
1	"acoustic"	1000
2	"alt-rock"	999
3	"cantopop"	999
4	"ambient"	999
5	"tango"	999
6	"afrobeat"	999
7	"chicago-house"	998
8	"disney"	998
9	"bluegrass"	998
10	"forro"	998

Challenges

Duplicate track_id
issue

Removed Duplicate

Multiple artist in
one field

Split artists

Slow Query
Performance

Used Batch processing

Lesson Learned & Next Step

1. Graph Database are powerful for relationship

2. Data Cleaning are critical

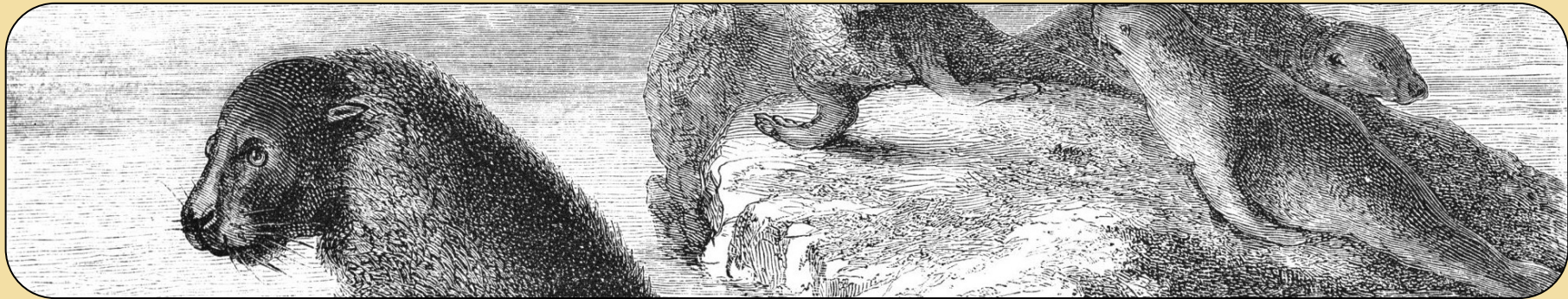
3. Query structure affects performance

1. Add more song features (tempo, vibe)

2. Improve Query Optimization

3 . Build Recommendation System

Q & A



THANK YOU

